BullMQ + Notifications

Corrected DB Schema · All Indexes · BullMQ Producers & Consu
Triggers · API Reference

BULLMQ

Complete BullMQ Queue Reference — All Producers & Consumers

Every queue name · producer MS · consumer MS · payload · when triggered

Architecture rule: The service that detects the event is the PRODUCER. The service that does the work is the CONSUMER. They communicate only through Redis/BullMQ — never direct HTTP for async work. Each queue name is a constant in @rsl/shared-constants.

All 15 BullMQ Queues

Queue Name (constant)	Producer MS	Consumer MS	Priority	When Triggered
deactivate-listings	transaction-service	listing-service	1 (highest)	Card sold on any platform. Must end all other platform listings immediately.
sale-notification	transaction-service	notification-service	1	SELL confirmed. Dealer must see sale notification within seconds.
want-list-match	inventory-service	notification-service	1	Dealer adds card that matches a consumer's want list.
offer-received	listing-service	notification-service	1	eBay best offer received on dealer's active listing.
price-alert-triggered	card-db-service	notification-service	1	Card comp price crosses user's target price threshold.
notify-send	notification-service	notification-service	2	Internal: route to FCM / email / in-app based on user preferences.
ai-narrative-publish	admin-service	notification-service	2	Admin approves narrative. Push to all users holding affected cards.
inventory-aging-alert	inventory-service	notification-service	2	BullMQ cron: daily 8am. Cards held 60+ days flagged.
narrative-generate	ai-narrative-service	ai-narrative-service	2	Score delta >= 15 detected. Queue Claude API call for narrative generation.
data-ingestion	ai-narrative-service	ai-narrative-service	3	BullMQ cron: every 2 hours. Fetch Sportradar, NewsAPI, Twitter, eBay.
analytics-snapshot	analytics-service	analytics-service	3	BullMQ cron: daily 2am. Pre-aggregate daily_summaries from transactions.
tax-report-generate	analytics-service	analytics-service	3	Dealer requests tax export. Heavy job runs async, delivers via S3 link.
offline-sync-drain	transaction-service	transaction-service	2	Mobile reconnects after offline period. Drain queued offline transactions.
platform-token-refresh	user-service	user-service	2	BullMQ cron: hourly. Refresh expiring eBay/Whatnot OAuth tokens.
email-send	notification-service	notification-service	3	Transactional emails: sale receipts, tax summaries, weekly digest.

Detailed Queue Payloads — What Data Flows Through Each Queue

Queue: deactivate-listings | *Priority 1* | *Producer: transaction-service* | *Consumer: listing-service*

```
// PRODUCER: transaction-service/src/handlers/sell.handler.ts
await deactivateListingsQueue.add('deactivate', {
```

```
  inventoryId:  'uuid',           // find all active listings for this inventory
  soldPlatform: 'ebay',          // don't deactivate the platform it sold on
  soldAt:       '2026-04-01T...',
  dealerId:     'uuid',
  cardName:     'Mahomes 2017 Prizm PSA 10',  // for logging
}, { priority: 1, attempts: 5, backoff: { type: 'exponential', delay: 2000 } })

// CONSUMER: listing-service/src/jobs/deactivate.worker.ts
worker.process(async (job) => {
  const { inventoryId, soldPlatform } = job.data
  const siblings = await db.select().from(listings)
    .where(and(eq(listings.inventoryId, inventoryId),
               eq(listings.status, 'active'),
               ne(listings.platform, soldPlatform)))
  for (const listing of siblings) {
    await callPlatformEndListingAPI(listing.platform, listing.platformListingId)
    await db.update(listings).set({ status: 'ended' }).where(eq(listings.id, listing.id))
  }
})
```

Queue: sale-notification | *Priority 1* | *Producer: transaction-service* | *Consumer: notification-service*

```
// PRODUCER: transaction-service/src/handlers/sell.handler.ts
await saleNotificationQueue.add('sale', {
  type:         'sale',
  userId:       'uuid',
  cardName:     'Mahomes 2017 Prizm PSA 10',
  salePrice:    341.00,
  costBasis:    280.00,
  profit:       61.00,
  profitPct:    21.8,
  platform:     'ebay',
  transactionId:'uuid',
}, { priority: 1, attempts: 3 })

// CONSUMER: notification-service/src/jobs/notify.worker.ts
worker.process(async (job) => {
  const { userId, cardName, profit } = job.data
  const prefs = await getUserPreferences(userId)
  if (prefs.saleAlerts) {
    await sendFCMPush(userId, {
      title: 'Card Sold!',
      body:  `${cardName} sold — Profit: $${profit}`,
      data:  { screen: 'transaction', id: job.data.transactionId }
    })
  }
})
```

Queue: narrative-generate | *Priority 2* | *Producer: ai-narrative-service* | *Consumer: ai-narrative-service*

```
// PRODUCER: ai-narrative-service/src/jobs/scoring.worker.ts
// Fires when CardPulse score delta >= 15
await narrativeGenerateQueue.add('generate', {
  playerName:      'Jayden Daniels',
  sport:           'Football',
  previousScore:   52.0,
  currentScore:    87.0,
```

```
  delta:           35.0,
  primaryDriver:   'performance',   // factor with largest delta
  narrativeType:   'breakout',
  factorScores: {
    performance: 95, sentiment: 78, event: 85,
    momentum: 80, liquidity: 65, volatility: 55
  },
  rawData: {
    newsArticles: [...],   // articles from this cycle
    sportradar:   {...},   // game stats
    ebayComps:    [...],   // recent sales
  },
  fetchWindow: { from: '2026-04-01T06:00:00Z', to: '2026-04-01T08:00:00Z' }
}, { priority: 2, attempts: 3 })

// CONSUMER: ai-narrative-service/src/jobs/generate.worker.ts
worker.process(async (job) => {
  const prompt = buildClaudePrompt(job.data)
  const response = await callClaudeAPI(prompt)
  await db.insert(narratives).values({
    playerName: job.data.playerName,
    headline:   response.headline,
    body:       response.body,
    status:     'pending_review',
    ...
  })
})
```

Queue: data-ingestion *| Priority 3 | Cron: every 2 hours | Both producer and consumer: ai-narrative-service*

```
// CRON SETUP: ai-narrative-service/src/jobs/ingestion.cron.ts
// Repeating job — BullMQ schedules itself
await dataIngestionQueue.add('ingest', {
  triggeredAt: new Date().toISOString(),
  manual: false   // true when triggered via /narratives/trigger-ingestion (dev only)
}, { repeat: { cron: '0 */2 * * *' } })

// CONSUMER: processes each player in watchlist
worker.process(async (job) => {
  const players = await db.select().from(playerWatchlist)
    .where(and(eq(playerWatchlist.active, true)))
  const batches = chunk(players, 10)
  for (const batch of batches) {
    await Promise.all(batch.map(p => processPlayer(p)))
    await sleep(1000)  // rate limit respect
  }
})
```

Queue: ai-narrative-publish *| Priority 2 | Producer: admin-service | Consumer: notification-service*

```
// PRODUCER: admin-service/src/handlers/narrative-approve.handler.ts
// Admin clicks Approve in admin panel
await narrativePublishQueue.add('publish', {
  narrativeId:  'uuid',
  playerName:   'Jayden Daniels',
  headline:     'Daniels rookies surge 18% after record game',
  shortSummary: 'PSA 10 Prizm Silver: $48 -> $58 in 48 hours',
```

```
  narrativeType:'breakout',
  priceChangePct: 18.0,
  affectedCardIds: ['card_id_1', 'card_id_2'],
}, { priority: 2, attempts: 3 })

// CONSUMER: notification-service/src/jobs/narrative-push.worker.ts
worker.process(async (job) => {
  const { playerName, affectedCardIds } = job.data
  // Find all users holding these cards in inventory OR collection
  const dealerUserIds = await db.selectDistinct({ userId: inventory.userId })
    .from(inventory).where(inArray(inventory.cardId, affectedCardIds))
  const consumerUserIds = await db.selectDistinct({ userId: consumerCollection.userId })
    .from(consumerCollection).where(inArray(consumerCollection.cardId, affectedCardIds))
  const allUserIds = [...new Set([...dealerUserIds, ...consumerUserIds].map(r=>r.userId))]
  // Send FCM batch (max 500 per Firebase call)
  const batches = chunk(allUserIds, 500)
  for (const batch of batches) {
    await sendFCMMulticast(batch, {
      title: `AI Insight: ${playerName}`,
      body:  job.data.shortSummary,
      data:  { screen: 'narrative', id: job.data.narrativeId }
    })
  }
})
```

Queue: price-alert-triggered *| Priority 1 | Producer: card-db-service | Consumer: notification-service*

```
// PRODUCER: card-db-service/src/jobs/price-alert-evaluator.ts
// BullMQ cron every 15 minutes — evaluates all active price alerts
const activeAlerts = await db.select().from(priceAlerts)
  .where(and(eq(priceAlerts.isActive, true), eq(priceAlerts.isTriggered, false)))
for (const alert of activeAlerts) {
  const currentComp = await getLatestComp(alert.cardId, alert.gradeKey)
  const triggered = alert.direction === 'above'
    ? currentComp >= alert.targetPrice
    : currentComp <= alert.targetPrice
  if (triggered) {
    await priceAlertQueue.add('triggered', {
      alertId:      alert.id,
      userId:       alert.userId,
      cardId:       alert.cardId,
      gradeKey:     alert.gradeKey,
      targetPrice:  alert.targetPrice,
      currentPrice: currentComp,
      direction:    alert.direction,
      narrativeId:  await getRelevantNarrative(alert.cardId), // attach AI context if available
    }, { priority: 1 })
    await db.update(priceAlerts).set({ isTriggered: true, triggeredAt: new Date() })
      .where(eq(priceAlerts.id, alert.id))
  }
}
```

Queue: want-list-match *| Priority 1 | Producer: inventory-service | Consumer: notification-service*

```
// PRODUCER: inventory-service/src/handlers/add-inventory.handler.ts
// Fires EVERY TIME a dealer adds a card to inventory
await wantListMatchQueue.add('match-check', {
```

```
  inventoryId: 'uuid',
  cardId:      'card_id',
  gradeKey:    'PSA_10',
  dealerId:    'uuid',
  dealerName:  'Mike Sherrer',
  price:       341.00,            // dealer's asking price / cost basis
  playerName:  'Mahomes',
}, { priority: 1, attempts: 3 })

// CONSUMER: notification-service/src/jobs/want-list.worker.ts
worker.process(async (job) => {
  const { cardId, gradeKey, price } = job.data
  const matches = await db.select().from(wantList)
    .where(and(eq(wantList.cardId, cardId),
               eq(wantList.gradeKey, gradeKey),
               lte(wantList.maxPrice, price)))  // price within their budget
  for (const match of matches) {
    await sendFCMPush(match.userId, {
      title: 'Want List Match Found!',
      body:  `${job.data.playerName} ${gradeKey} available from ${job.data.dealerName}`,
      data:  { screen: 'dealer', id: job.data.dealerId }
    })
  }
})
```

Queue: analytics-snapshot *| Priority 3 | Cron: 2am daily | Both: analytics-service*

```
// CRON: analytics-service/src/jobs/snapshot.cron.ts
await snapshotQueue.add('daily-snapshot', {
  date: new Date().toISOString().split('T')[0],  // YYYY-MM-DD
  manual: false
}, { repeat: { cron: '0 2 * * *' } })

// CONSUMER: analytics-service/src/jobs/snapshot.worker.ts
worker.process(async (job) => {
  const { date } = job.data
  // Run on READ REPLICA only
  const results = await dbRead.execute(sql`
    SELECT user_id,
           COUNT(*) FILTER (WHERE type='buy') as cards_bought,
           COUNT(*) FILTER (WHERE type='sell') as cards_sold,
           SUM(price) FILTER (WHERE type='buy') as total_spent,
           SUM(price) FILTER (WHERE type='sell') as total_revenue,
           SUM(profit) FILTER (WHERE type='sell') as net_profit,
           jsonb_object_agg(channel, channel_revenue) as revenue_by_channel
    FROM transactions WHERE DATE(created_at) = ${date}
    GROUP BY user_id
  `)
  await dbWrite.insert(dailySummaries).values(results).onConflictDoUpdate(...)
})
```

Queue: tax-report-generate *| Priority 3 | Producer: analytics-service | Consumer: analytics-service*

```
// PRODUCER: analytics-service/src/handlers/tax-export.handler.ts
// Called when dealer requests tax report (never synchronous)
const jobId = await taxReportQueue.add('generate', {
  userId:   'uuid',
```

```
    taxYear:  2025,
    format:   'pdf',   // pdf | csv
    email:    'dealer@example.com',
  }, { priority: 3, attempts: 2 })
  // Return jobId immediately to client - don't make them wait
  return reply.send({ jobId, status: 'queued', message: 'Report generating. Download link sent via notification.' })

  // CONSUMER: analytics-service/src/jobs/tax-report.worker.ts
  worker.process(async (job) => {
    const report = await generateTaxReport(job.data.userId, job.data.taxYear)
    const s3Url = await uploadToS3(report, `tax-reports/${job.data.userId}/${job.data.taxYear}.pdf`)
    // Notify dealer report is ready
    await notifyQueue.add('tax-ready', {
      type: 'tax_report_ready',
      userId: job.data.userId,
      downloadUrl: s3Url,
      expiresIn: '24h'
    })
  })
```

NOTIFICATIONS

Complete Notification Trigger Map

Every notification type · where it fires · what it says · which channel

Notification channels: push (Firebase FCM) | email (Resend) | in_app (stored in notifications table, shown in bell icon). User preferences control which channels are active per type. Quiet hours respected for non-critical notifications.

Notification Type	Triggered In	BullMQ Queue	Channel	Title Example	Body Example	Priority
sale	transaction-service SELL handler	sale-notification	push + in_app	Card Sold!	Mahomes PSA 10 sold — Profit: $61 (21.8%)	Critical
sale (platform)	listing-service webhook receiver	sale-notification	push + in_app	Sold on eBay	Mahomes PSA 10 sold on eBay for $341	Critical
offer_received	listing-service webhook receiver	offer-received	push + in_app	Offer Received	$310 offer on Mahomes PSA 10 on eBay	High
price_alert	card-db-service cron (15min)	price-alert-triggered	push + in_app	Price Alert Hit!	Mahomes PSA 10 dropped below $300	High
want_list_match	inventory-service add card	want-list-match	push + in_app	Want List Match!	Mahomes PSA 10 available from Mike Sherrer	High
ai_narrative	admin-service approve narrative	ai-narrative-publish	push + in_app	AI Insight: Daniels	Daniels rookies +18% after record game	Medium
aging_alert	inventory-service cron (daily 8am)	inventory-aging-alert	push + in_app	Card Aging Alert	Trout PSA 9 held 67 days — consider listing	Medium
inventory_trending	ai-narrative-service scoring	ai-narrative-publish	in_app only	Your Inventory Moving	2 of your Daniels cards up 18% today	Medium
show_reminder	notification-service cron (24hr before)	notify-send	push + in_app	Show Tomorrow!	Dallas Card Expo starts 9am tomorrow	Low
new_dealer_inventory	inventory-service add card	want-list-match	push	Mike Sherrer Added Cards	12 new cards including Mahomes, Judge	Low
weekly_digest	analytics-service cron (Monday 9am)	email-send	email only	Your Week in Review	You made $342 profit on 8 sales this week	Low
tax_report_ready	analytics-service worker	notify-send	push + email	Tax Report Ready	Your 2025 tax report is ready. Download link expires in 24h	Medium
platform_alert	listing-service polling	notify-send	in_app only	Listing Undercut	Competitor listed same card $20 cheaper on eBay	Low
system	admin-service	notify-send	push	System Update	RSL Cards updated with new features	Low

Notification Preference Check Logic

```
// notification-service/src/utils/preference-check.ts
// Run this BEFORE sending any notification
export async function shouldSendNotification(
  userId: string, type: NotifType, channel: NotifChannel
): Promise<boolean> {
  const prefs = await getUserPreferences(userId)

  // 1. Check if this notification type is enabled
  if (!prefs[type]) return false
```

```
  // 2. Check quiet hours (don't send push during 10pm-8am user local time)
  if (channel === 'push' && isQuietHours(prefs.quietHoursStart, prefs.quietHoursEnd, prefs.timezone)) {
    // Queue for delivery after quiet hours end (unless critical)
    if (!CRITICAL_TYPES.includes(type)) return false
  }

  // 3. Check frequency limits (max X notifications per day for non-critical)
  if (!CRITICAL_TYPES.includes(type)) {
    const todayCount = await getTodayNotificationCount(userId)
    if (todayCount >= prefs.dailyLimit) return false
  }

  return true
}

const CRITICAL_TYPES = ['sale', 'offer_received', 'price_alert', 'want_list_match']
```

SCHEMA

Corrected & Complete DB Schema — All 30+ Tables

All fixes applied: comps tables corrected, AI snapshot tables added, indexes included

1. Auth Service — shared/db/schema/auth.ts

```
import { pgTable, uuid, varchar, boolean, timestamp, pgEnum } from 'drizzle-orm/pg-core'

export const roleEnum         = pgEnum('role', ['dealer','consumer','admin'])
export const oauthProviderEnum = pgEnum('oauth_provider', ['google','apple'])

export const users = pgTable('users', {
  id:                 uuid('id').primaryKey().defaultRandom(),
  email:              varchar('email', { length: 255 }).unique().notNull(),
  passwordHash:       varchar('password_hash', { length: 255 }),     // null = OAuth only
  role:               roleEnum('role').default('consumer').notNull(),
  oauthProvider:      oauthProviderEnum('oauth_provider'),
  oauthId:            varchar('oauth_id', { length: 255 }),
  twoFactorSecret:    varchar('two_factor_secret', { length: 255 }), // AES-256-GCM encrypted
  twoFactorEnabled:   boolean('two_factor_enabled').default(false),
  isEmailVerified:    boolean('is_email_verified').default(false),
  isActive:           boolean('is_active').default(true),
  lastLoginAt:        timestamp('last_login_at', { withTimezone: true }),
  createdAt:          timestamp('created_at', { withTimezone: true }).defaultNow(),
  updatedAt:          timestamp('updated_at', { withTimezone: true }).defaultNow(),
})

export const refreshTokens = pgTable('refresh_tokens', {
  id:         uuid('id').primaryKey().defaultRandom(),
  userId:     uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).notNull(),
  tokenHash:  varchar('token_hash', { length: 255 }).notNull().unique(),
  deviceInfo: varchar('device_info', { length: 500 }),
  ipAddress:  varchar('ip_address', { length: 50 }),
  expiresAt:  timestamp('expires_at', { withTimezone: true }).notNull(),
  createdAt:  timestamp('created_at', { withTimezone: true }).defaultNow(),
})

export const deviceTokens = pgTable('device_tokens', {
  id:        uuid('id').primaryKey().defaultRandom(),
  userId:    uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).notNull(),
  fcmToken:  varchar('fcm_token', { length: 500 }).notNull(),
  platform:  varchar('platform', { length: 10 }).notNull(),   // ios | android
  deviceId:  varchar('device_id', { length: 255 }),
  createdAt: timestamp('created_at', { withTimezone: true }).defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).defaultNow(),
})
```

2. User Service — shared/db/schema/user.ts

```
import { pgTable, uuid, varchar, text, boolean, timestamp, integer } from 'drizzle-orm/pg-core'
import { users } from './auth'

export const dealerProfiles = pgTable('dealer_profiles', {
  id:               uuid('id').primaryKey().defaultRandom(),
  userId:           uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).unique().notNull(),
```

```
  displayName:      varchar('display_name', { length: 255 }).notNull(),
  bio:              text('bio'),
  phone:            varchar('phone', { length: 20 }),
  photoUrl:         varchar('photo_url', { length: 500 }),
  sports:           text('sports').array(),         // ['football','baseball']
  sellChannels:     text('sell_channels').array(),  // ['card_shows','ebay']
  customUrl:        varchar('custom_url', { length: 100 }).unique(), // rslcards.com/dealers/name
  isPublic:         boolean('is_public').default(true),
  subscriptionPlan: varchar('subscription_plan', { length: 50 }).default('free'),
  rating:           varchar('rating', { length: 5 }).default('0'),   // avg dealer rating
  reviewCount:      integer('review_count').default(0),
  followerCount:    integer('follower_count').default(0),
  createdAt:        timestamp('created_at', { withTimezone: true }).defaultNow(),
  updatedAt:        timestamp('updated_at', { withTimezone: true }).defaultNow(),
})

export const consumerProfiles = pgTable('consumer_profiles', {
  id:          uuid('id').primaryKey().defaultRandom(),
  userId:      uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).unique().notNull(),
  displayName: varchar('display_name', { length: 255 }).notNull(),
  photoUrl:    varchar('photo_url', { length: 500 }),
  sports:      text('sports').array(),
  teams:       text('teams').array(),
  players:     text('players').array(),
  isPremium:   boolean('is_premium').default(false),
  createdAt:   timestamp('created_at', { withTimezone: true }).defaultNow(),
  updatedAt:   timestamp('updated_at', { withTimezone: true }).defaultNow(),
})

export const paymentMethods = pgTable('payment_methods', {
  id:          uuid('id').primaryKey().defaultRandom(),
  userId:      uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).notNull(),
  type:        varchar('type', { length: 20 }).notNull(), // venmo|zelle|paypal|cashapp|other
  handle:      varchar('handle', { length: 255 }).notNull(),
  isDefault:   boolean('is_default').default(false),
  createdAt:   timestamp('created_at', { withTimezone: true }).defaultNow(),
})

export const platformConnections = pgTable('platform_connections', {
  id:              uuid('id').primaryKey().defaultRandom(),
  userId:          uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).notNull(),
  platform:        varchar('platform', { length: 50 }).notNull(), // ebay|whatnot|mercari|tcgplayer
  accessToken:     text('access_token'),           // encrypted at rest
  refreshToken:    text('refresh_token'),          // encrypted at rest
  tokenExpiresAt:  timestamp('token_expires_at', { withTimezone: true }),
  platformUserId:  varchar('platform_user_id', { length: 255 }),
  isActive:        boolean('is_active').default(true),
  createdAt:       timestamp('created_at', { withTimezone: true }).defaultNow(),
  updatedAt:       timestamp('updated_at', { withTimezone: true }).defaultNow(),
})

export const userPreferences = pgTable('user_preferences', {
  id:               uuid('id').primaryKey().defaultRandom(),
  userId:           uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).unique().notNull(),
```

```
  saleAlerts:       boolean('sale_alerts').default(true),
  priceAlerts:      boolean('price_alerts').default(true),
  aiNarratives:     boolean('ai_narratives').default(true),
  agingAlerts:      boolean('aging_alerts').default(true),
  showReminders:    boolean('show_reminders').default(true),
  wantListMatches:  boolean('want_list_matches').default(true),
  weeklyDigest:     boolean('weekly_digest').default(true),
  quietHoursStart:  varchar('quiet_hours_start', { length: 5 }).default('22:00'),
  quietHoursEnd:    varchar('quiet_hours_end', { length: 5 }).default('08:00'),
  timezone:         varchar('timezone', { length: 50 }).default('America/New_York'),
  dailyLimit:       integer('daily_limit').default(20),
  updatedAt:        timestamp('updated_at', { withTimezone: true }).defaultNow(),
})

export const dealerFollowers = pgTable('dealer_followers', {
  id:         uuid('id').primaryKey().defaultRandom(),
  dealerId:   uuid('dealer_id').references(() => users.id, { onDelete: 'cascade' }).notNull(),
  followerId: uuid('follower_id').references(() => users.id, { onDelete: 'cascade' }).notNull(),
  createdAt:  timestamp('created_at', { withTimezone: true }).defaultNow(),
})

export const customers = pgTable('customers', {
  id:               uuid('id').primaryKey().defaultRandom(),
  userId:           uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).notNull(), // dealer
  name:             varchar('name', { length: 255 }).notNull(),
  phone:            varchar('phone', { length: 20 }),
  email:            varchar('email', { length: 255 }),
  notes:            text('notes'),
  isFavorite:       boolean('is_favorite').default(false),
  totalTransactions:integer('total_transactions').default(0),
  totalSpent:       varchar('total_spent', { length: 20 }).default('0'), // decimal as string
  lastSeenAt:       timestamp('last_seen_at', { withTimezone: true }),
  createdAt:        timestamp('created_at', { withTimezone: true }).defaultNow(),
})
```

3. Inventory Service — shared/db/schema/inventory.ts

```
import { pgTable, uuid, varchar, decimal, integer, boolean, timestamp, text, pgEnum } from 'drizzle-orm/pg-core'
import { users } from './auth'

export const listingStatusEnum = pgEnum('listing_status', ['unlisted','listed','sold','archived'])
export const gradeCompanyEnum  = pgEnum('grade_company', ['PSA','BGS','SGC','CSG','RAW'])

export const inventory = pgTable('inventory', {
  id:                   uuid('id').primaryKey().defaultRandom(),
  userId:               uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).notNull(),
  cardId:               varchar('card_id', { length: 255 }),           // FK to cards table
  playerName:           varchar('player_name', { length: 255 }).notNull(),
  year:                 integer('year'),
  setName:              varchar('set_name', { length: 255 }),
  variation:            varchar('variation', { length: 255 }),
  cardNumber:           varchar('card_number', { length: 50 }),
  sport:                varchar('sport', { length: 50 }),
  gradeCompany:         gradeCompanyEnum('grade_company'),             // PSA | BGS | SGC | RAW
  gradeValue:           varchar('grade_value', { length: 10 }),        // 10 | 9.5 | 9 | NM-MT
```

```
  gradeKey:             varchar('grade_key', { length: 30 }),          // PSA_10 | BGS_9.5 | RAW
  certNumber:           varchar('cert_number', { length: 50 }),        // PSA/BGS cert for barcode scan
  costBasis:            decimal('cost_basis', { precision: 10, scale: 2 }).notNull(),
  currentMarketValue:   decimal('current_market_value', { precision: 10, scale: 2 }),
  unrealizedGain:       decimal('unrealized_gain', { precision: 10, scale: 2 }),
  quantity:             integer('quantity').default(1),
  isConsignment:        boolean('is_consignment').default(false),
  consignmentOwner:     varchar('consignment_owner', { length: 255 }),
  consignmentCommPct:   decimal('consignment_comm_pct', { precision: 5, scale: 2 }),
  listedPlatforms:      text('listed_platforms').array(),              // ['ebay','whatnot']
  listingStatus:        listingStatusEnum('listing_status').default('unlisted'),
  photos:               text('photos').array(),                        // S3 URLs
  notes:                text('notes'),
  addedAt:              timestamp('added_at', { withTimezone: true }).defaultNow(),
  updatedAt:            timestamp('updated_at', { withTimezone: true }).defaultNow(),
})

export const bulkPurchases = pgTable('bulk_purchases', {
  id:           uuid('id').primaryKey().defaultRandom(),
  userId:       uuid('user_id').references(() => users.id).notNull(),
  totalPrice:   decimal('total_price', { precision: 10, scale: 2 }).notNull(),
  itemCount:    integer('item_count').notNull(),
  paymentMethod:varchar('payment_method', { length: 50 }),
  notes:        text('notes'),
  createdAt:    timestamp('created_at', { withTimezone: true }).defaultNow(),
})
```

4. Transaction Service — shared/db/schema/transaction.ts

```
import { pgTable, uuid, varchar, decimal, boolean, timestamp, text, pgEnum } from 'drizzle-orm/pg-core'
import { users } from './auth'
import { inventory } from './inventory'
import { customers } from './user'

export const txTypeEnum      = pgEnum('tx_type', ['buy','sell','trade'])
export const txChannelEnum   = pgEnum('tx_channel', ['card_show','ebay','whatnot','mercari',
                                'tcgplayer','facebook','shopify','comc','goldin','app','other'])
export const paymentMethodEnum = pgEnum('payment_method_type',
                                ['cash','venmo','zelle','paypal','cashapp','trade','other'])
export const dealRatingEnum  = pgEnum('deal_rating', ['good_deal','fair_price','overpaying'])

export const transactions = pgTable('transactions', {
  id:             uuid('id').primaryKey().defaultRandom(),
  userId:         uuid('user_id').references(() => users.id).notNull(),
  inventoryId:    uuid('inventory_id').references(() => inventory.id),
  customerId:     uuid('customer_id').references(() => customers.id),
  type:           txTypeEnum('type').notNull(),
  channel:        txChannelEnum('channel').default('card_show'),
  price:          decimal('price', { precision: 10, scale: 2 }).notNull(),
  costBasis:      decimal('cost_basis', { precision: 10, scale: 2 }),
  profit:         decimal('profit', { precision: 10, scale: 2 }),
  profitPct:      decimal('profit_pct', { precision: 8, scale: 2 }),
  platformFee:    decimal('platform_fee', { precision: 10, scale: 2 }),
  netToDealer:    decimal('net_to_dealer', { precision: 10, scale: 2 }),
  paymentMethod:  paymentMethodEnum('payment_method'),
```

```
  dealRating:      dealRatingEnum('deal_rating'),
  compPriceAtTime: decimal('comp_price_at_time', { precision: 10, scale: 2 }), // snapshot
  playerName:      varchar('player_name', { length: 255 }),   // denormalized for reports
  gradeKey:        varchar('grade_key', { length: 30 }),       // denormalized for reports
  cardSnapshot:    text('card_snapshot'),                       // full JSON card at time of tx
  isOffline:       boolean('is_offline').default(false),
  localId:         varchar('local_id', { length: 255 }).unique(), // mobile SQLite UUID dedup
  createdAt:       timestamp('created_at', { withTimezone: true }).defaultNow(),
})

export const tradeItems = pgTable('trade_items', {
  id:             uuid('id').primaryKey().defaultRandom(),
  transactionId:  uuid('transaction_id').references(() => transactions.id, { onDelete: 'cascade' }).notNull(),
  direction:      varchar('direction', { length: 10 }).notNull(), // given | received
  inventoryId:    uuid('inventory_id').references(() => inventory.id),
  playerName:     varchar('player_name', { length: 255 }),
  gradeKey:       varchar('grade_key', { length: 30 }),
  marketValue:    decimal('market_value', { precision: 10, scale: 2 }),
  cashAdjustment: decimal('cash_adjustment', { precision: 10, scale: 2 }).default('0'),
})

export const offlineSyncQueue = pgTable('offline_sync_queue', {
  id:        uuid('id').primaryKey().defaultRandom(),
  userId:    uuid('user_id').references(() => users.id).notNull(),
  localId:   varchar('local_id', { length: 255 }).notNull().unique(),
  type:      txTypeEnum('type').notNull(),
  payload:   text('payload').notNull(),   // full JSON transaction payload
  synced:    boolean('synced').default(false),
  syncedAt:  timestamp('synced_at', { withTimezone: true }),
  createdAt: timestamp('created_at', { withTimezone: true }).defaultNow(),
})
```

5. Listing Service — shared/db/schema/listing.ts [CORRECTED]

> FIX APPLIED: platformPriceComps removed from listing service. It now belongs in card-db-service as cardCompSnapshots with gradeKey added.

```
import { pgTable, uuid, varchar, decimal, text, boolean, timestamp, pgEnum } from 'drizzle-orm/pg-core'
import { users } from './auth'
import { inventory } from './inventory'

export const listingPlatformEnum = pgEnum('listing_platform', [
  'ebay','whatnot','mercari','tcgplayer','shopify','comc','facebook','goldin','myslabs','instagram'
])
export const listingStatusEnum = pgEnum('platform_listing_status',
  ['draft','pending','active','sold','ended','failed'])

export const listings = pgTable('listings', {
  id:                uuid('id').primaryKey().defaultRandom(),
  inventoryId:       uuid('inventory_id').references(() => inventory.id).notNull(),
  userId:            uuid('user_id').references(() => users.id).notNull(),
  platform:          listingPlatformEnum('platform').notNull(),
  platformListingId: varchar('platform_listing_id', { length: 255 }),  // eBay itemId etc
  status:            listingStatusEnum('status').default('draft'),
  listPrice:         decimal('list_price', { precision: 10, scale: 2 }).notNull(),
```

```
  platformFeePct:    decimal('platform_fee_pct', { precision: 5, scale: 2 }),
  platformFeeAmt:    decimal('platform_fee_amt', { precision: 10, scale: 2 }),
  estimatedShipping: decimal('estimated_shipping', { precision: 10, scale: 2 }),
  netToDealer:       decimal('net_to_dealer', { precision: 10, scale: 2 }),
  title:             text('title'),
  description:       text('description'),
  photos:            text('photos').array(),
  scheduledAt:       timestamp('scheduled_at', { withTimezone: true }),
  listedAt:          timestamp('listed_at', { withTimezone: true }),
  soldAt:            timestamp('sold_at', { withTimezone: true }),
  soldPrice:         decimal('sold_price', { precision: 10, scale: 2 }),
  errorMessage:      text('error_message'),
  createdAt:         timestamp('created_at', { withTimezone: true }).defaultNow(),
  updatedAt:         timestamp('updated_at', { withTimezone: true }).defaultNow(),
})
```

6. Card DB Service — shared/db/schema/carddb.ts [MAJOR CORRECTIONS]

> THREE FIXES APPLIED: (1) ebayRecentSales renamed to platformSoldListings — now covers ALL platforms, not just eBay. (2) cardCompSnapshots added with gradeKey — one row per card+grade+platform, UPSERTED each fetch. (3) contentHash added to platformSoldListings for deduplication across API calls.

```
import { pgTable, uuid, varchar, decimal, integer, text, timestamp, boolean, uniqueIndex } from 'drizzle-orm/pg-cor
e'
import { users } from './auth'
import { listingPlatformEnum } from './listing'

// Core card catalog — populated from Ximilar + TCDB
export const cards = pgTable('cards', {
  id:            varchar('id', { length: 255 }).primaryKey(),  // ximilar_id or tcdb_id
  playerName:    varchar('player_name', { length: 255 }).notNull(),
  year:          integer('year'),
  setName:       varchar('set_name', { length: 255 }),
  variation:     varchar('variation', { length: 255 }),
  cardNumber:    varchar('card_number', { length: 50 }),
  sport:         varchar('sport', { length: 50 }),
  manufacturer:  varchar('manufacturer', { length: 100 }),  // Topps, Panini, Upper Deck
  isRookie:      boolean('is_rookie').default(false),
  isAutograph:   boolean('is_autograph').default(false),
  isRelic:       boolean('is_relic').default(false),
  printRun:      integer('print_run'),
  stockImageUrl: varchar('stock_image_url', { length: 500 }),
  source:        varchar('source', { length: 50 }),          // ximilar | tcdb | manual
  createdAt:     timestamp('created_at', { withTimezone: true }).defaultNow(),
  updatedAt:     timestamp('updated_at', { withTimezone: true }).defaultNow(),
})

// FIX 1: ALL platforms, not just eBay. gradeKey added. contentHash for dedup.
export const platformSoldListings = pgTable('platform_sold_listings', {
  id:             uuid('id').primaryKey().defaultRandom(),
  cardId:         varchar('card_id', { length: 255 }).references(() => cards.id).notNull(),
  gradeKey:       varchar('grade_key', { length: 30 }).notNull(), // PSA_10 | BGS_9.5 | RAW
  platform:       listingPlatformEnum('platform').notNull(),       // ebay | whatnot | mercari | comc
  soldPrice:      decimal('sold_price', { precision: 10, scale: 2 }).notNull(),
  platformItemId: varchar('platform_item_id', { length: 255 }),    // eBay itemId, Whatnot listingId
```

```
  soldAt:         timestamp('sold_at', { withTimezone: true }).notNull(),
  title:          varchar('title', { length: 500 }),
  condition:      varchar('condition', { length: 100 }),
  contentHash:    varchar('content_hash', { length: 64 }).unique(), // MD5(platform+itemId+soldAt)
  createdAt:      timestamp('created_at', { withTimezone: true }).defaultNow(),
})

// FIX 2: cardCompSnapshots replaces platformPriceComps from listing service.
// One row per card+gradeKey+platform. UPSERTED every 15min cache cycle.
// This drives the BUY screen cross-platform comparison.
export const cardCompSnapshots = pgTable('card_comp_snapshots', {
  id:             uuid('id').primaryKey().defaultRandom(),
  cardId:         varchar('card_id', { length: 255 }).references(() => cards.id).notNull(),
  gradeKey:       varchar('grade_key', { length: 30 }).notNull(), // CRITICAL: PSA_10 | BGS_9.5 | RAW
  platform:       listingPlatformEnum('platform').notNull(),
  avgSoldPrice:   decimal('avg_sold_price', { precision: 10, scale: 2 }),
  lastSoldPrice:  decimal('last_sold_price', { precision: 10, scale: 2 }),
  lowestActive:   decimal('lowest_active', { precision: 10, scale: 2 }),   // lowest current listing
  salesCount30d:  integer('sales_count_30d').default(0),
  priceTrend30d:  decimal('price_trend_30d', { precision: 8, scale: 2 }),  // % vs 30 days ago
  fetchedAt:      timestamp('fetched_at', { withTimezone: true }).defaultNow(),
}, (t) => ({
  uniqCompSnapshot: uniqueIndex('uq_comp_card_grade_platform').on(t.cardId, t.gradeKey, t.platform),
}))

// 30/90/365 day price history per card+grade (for sparkline charts)
export const cardPriceHistory = pgTable('card_price_history', {
  id:           uuid('id').primaryKey().defaultRandom(),
  cardId:       varchar('card_id', { length: 255 }).references(() => cards.id).notNull(),
  gradeKey:     varchar('grade_key', { length: 30 }).notNull(),
  avgSoldPrice: decimal('avg_sold_price', { precision: 10, scale: 2 }),
  minSoldPrice: decimal('min_sold_price', { precision: 10, scale: 2 }),
  maxSoldPrice: decimal('max_sold_price', { precision: 10, scale: 2 }),
  salesCount:   integer('sales_count'),
  priceTrend:   decimal('price_trend', { precision: 8, scale: 2 }),
  recordedDate: timestamp('recorded_date', { withTimezone: true }).defaultNow(),
})

// Consumer price alerts
export const priceAlerts = pgTable('price_alerts', {
  id:           uuid('id').primaryKey().defaultRandom(),
  userId:       uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).notNull(),
  cardId:       varchar('card_id', { length: 255 }).references(() => cards.id).notNull(),
  gradeKey:     varchar('grade_key', { length: 30 }).notNull(), // MUST have grade on alert
  targetPrice:  decimal('target_price', { precision: 10, scale: 2 }).notNull(),
  direction:    varchar('direction', { length: 10 }).default('below'), // below | above
  isTriggered:  boolean('is_triggered').default(false),
  triggeredAt:  timestamp('triggered_at', { withTimezone: true }),
  isActive:     boolean('is_active').default(true),
  createdAt:    timestamp('created_at', { withTimezone: true }).defaultNow(),
})

// Consumer want list
export const wantList = pgTable('want_list', {
```

```
  id:        uuid('id').primaryKey().defaultRandom(),
  userId:    uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).notNull(),
  cardId:    varchar('card_id', { length: 255 }).references(() => cards.id).notNull(),
  gradeKey:  varchar('grade_key', { length: 30 }),
  maxPrice:  decimal('max_price', { precision: 10, scale: 2 }),
  createdAt: timestamp('created_at', { withTimezone: true }).defaultNow(),
})

// Consumer collection
export const consumerCollection = pgTable('consumer_collection', {
  id:           uuid('id').primaryKey().defaultRandom(),
  userId:       uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).notNull(),
  cardId:       varchar('card_id', { length: 255 }).references(() => cards.id).notNull(),
  gradeKey:     varchar('grade_key', { length: 30 }).notNull(),
  costBasis:    decimal('cost_basis', { precision: 10, scale: 2 }),
  currentValue: decimal('current_value', { precision: 10, scale: 2 }),
  quantity:     integer('quantity').default(1),
  acquiredAt:   timestamp('acquired_at', { withTimezone: true }).defaultNow(),
  createdAt:    timestamp('created_at', { withTimezone: true }).defaultNow(),
})
```

7. AI Narrative Service — shared/db/schema/narrative.ts [MISSING TABLES ADDED]

> THREE TABLES ADDED: playerWatchlist (3-tier monitoring), playerSnapshots (memory for cycle comparison, UPSERTED), playerSnapshotHistory (append-only audit trail). These were completely missing from the original schema.

```
import { pgTable, uuid, varchar, text, decimal, boolean, timestamp, integer, pgEnum,
         uniqueIndex } from 'drizzle-orm/pg-core'

export const narrativeTypeEnum   = pgEnum('narrative_type',
  ['breakout','injury','hype','decline','seasonal','trade','hof','award','auction_record'])
export const narrativeStatusEnum = pgEnum('narrative_status',
  ['pending_review','approved','published','rejected'])
export const watchlistTierEnum   = pgEnum('watchlist_tier', ['core','subscribed','on_demand'])

export const narratives = pgTable('narratives', {
  id:               uuid('id').primaryKey().defaultRandom(),
  playerName:       varchar('player_name', { length: 255 }).notNull(),
  sport:            varchar('sport', { length: 50 }),
  cardIds:          text('card_ids').array(),          // affected card IDs
  headline:         varchar('headline', { length: 500 }).notNull(),
  shortSummary:     varchar('short_summary', { length: 280 }), // one-line for BUY screen
  body:             text('body').notNull(),
  whyItMatters:     text('why_it_matters'),            // dealer-specific insight
  narrativeType:    narrativeTypeEnum('narrative_type').notNull(),
  priceChangePct:   decimal('price_change_pct', { precision: 5, scale: 2 }),
  priceDirection:   varchar('price_direction', { length: 5 }),  // up | down
  correlatedEvents: text('correlated_events'),         // JSON: [{event, score}]
  status:           narrativeStatusEnum('status').default('pending_review'),
  reviewedBy:       uuid('reviewed_by'),
  publishedAt:      timestamp('published_at', { withTimezone: true }),
  createdAt:        timestamp('created_at', { withTimezone: true }).defaultNow(),
})

// NEW TABLE 1: 3-tier player monitoring list
```

```
export const playerWatchlist = pgTable('player_watchlist', {
  id:            uuid('id').primaryKey().defaultRandom(),
  playerName:    varchar('player_name', { length: 255 }).notNull().unique(),
  sport:         varchar('sport', { length: 50 }).notNull(),
  tier:          watchlistTierEnum('tier').notNull(),       // core | subscribed | on_demand
  holderCount:   integer('holder_count').default(0),        // users holding this player's cards
  active:        boolean('active').default(true),           // false when holderCount = 0
  lastCheckedAt: timestamp('last_checked_at', { withTimezone: true }),
  createdAt:     timestamp('created_at', { withTimezone: true }).defaultNow(),
})

// NEW TABLE 2: Current snapshot per player — UPSERTED every 2-hour cycle
// This IS the memory. Next cycle loads this to know what changed.
export const playerSnapshots = pgTable('player_snapshots', {
  id:                uuid('id').primaryKey().defaultRandom(),
  playerName:        varchar('player_name', { length: 255 }).notNull().unique(),
  sport:             varchar('sport', { length: 50 }),
  // CardPulse 6-factor scores (0-100 each)
  performanceScore:  decimal('performance_score', { precision: 5, scale: 2 }),
  sentimentScore:    decimal('sentiment_score', { precision: 5, scale: 2 }),
  eventScore:        decimal('event_score', { precision: 5, scale: 2 }),
  momentumScore:     decimal('momentum_score', { precision: 5, scale: 2 }),
  liquidityScore:    decimal('liquidity_score', { precision: 5, scale: 2 }),
  volatilityScore:   decimal('volatility_score', { precision: 5, scale: 2 }),
  totalScore:        decimal('total_score', { precision: 5, scale: 2 }),
  // Raw API data — carry-forward on API failure
  rawPerformance:    text('raw_performance'),  // JSON: Sportradar response
  rawSentiment:      text('raw_sentiment'),    // JSON: Twitter + Reddit
  rawEvents:         text('raw_events'),       // JSON: NewsAPI articles
  rawComps:          text('raw_comps'),        // JSON: eBay comps
  // KEY FIELD: used as fromTime filter in next cycle API calls
  lastFetchedAt:     timestamp('last_fetched_at', { withTimezone: true }),
  narrativeGenerated:boolean('narrative_generated').default(false),
  updatedAt:         timestamp('updated_at', { withTimezone: true }).defaultNow(),
})

// NEW TABLE 3: Append-only history — never updated/deleted
export const playerSnapshotHistory = pgTable('player_snapshot_history', {
  id:               uuid('id').primaryKey().defaultRandom(),
  playerName:       varchar('player_name', { length: 255 }).notNull(),
  sport:            varchar('sport', { length: 50 }),
  totalScore:       decimal('total_score', { precision: 5, scale: 2 }),
  performanceScore: decimal('performance_score', { precision: 5, scale: 2 }),
  sentimentScore:   decimal('sentiment_score', { precision: 5, scale: 2 }),
  eventScore:       decimal('event_score', { precision: 5, scale: 2 }),
  momentumScore:    decimal('momentum_score', { precision: 5, scale: 2 }),
  liquidityScore:   decimal('liquidity_score', { precision: 5, scale: 2 }),
  volatilityScore:  decimal('volatility_score', { precision: 5, scale: 2 }),
  fetchedFrom:      timestamp('fetched_from', { withTimezone: true }), // window start
  fetchedTo:        timestamp('fetched_to', { withTimezone: true }),   // window end
  createdAt:        timestamp('created_at', { withTimezone: true }).defaultNow(),
})

export const priceAnomalies = pgTable('price_anomalies', {
```

```
  id:             uuid('id').primaryKey().defaultRandom(),
  cardId:         varchar('card_id', { length: 255 }).notNull(),
  playerName:     varchar('player_name', { length: 255 }),
  priceChangePct: decimal('price_change_pct', { precision: 5, scale: 2 }),
  windowHours:    integer('window_hours').default(48),
  narrativeId:    uuid('narrative_id').references(() => narratives.id),
  processed:      boolean('processed').default(false),
  detectedAt:     timestamp('detected_at', { withTimezone: true }).defaultNow(),
})

export const contentCalendar = pgTable('content_calendar', {
  id:           uuid('id').primaryKey().defaultRandom(),
  title:        varchar('title', { length: 255 }).notNull(),
  eventType:    varchar('event_type', { length: 100 }),  // nfl_draft|topps_chrome|nba_finals
  sport:        varchar('sport', { length: 50 }),
  scheduledFor: timestamp('scheduled_for', { withTimezone: true }).notNull(),
  notes:        text('notes'),
  isActive:     boolean('is_active').default(true),
})
```

8. Notification Service — shared/db/schema/notification.ts

```
import { pgTable, uuid, varchar, text, boolean, timestamp, integer, pgEnum } from 'drizzle-orm/pg-core'
import { users } from './auth'

export const notifTypeEnum = pgEnum('notif_type', [
  'sale','price_alert','ai_narrative','show_reminder','aging_alert',
  'offer_received','inventory_trending','want_list_match','platform_alert',
  'weekly_digest','new_dealer_inventory','tax_report_ready','system'
])
export const notifChannelEnum = pgEnum('notif_channel', ['push','email','in_app'])
export const notifStatusEnum  = pgEnum('notif_status', ['pending','sent','failed','read'])

export const notifications = pgTable('notifications', {
  id:        uuid('id').primaryKey().defaultRandom(),
  userId:    uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).notNull(),
  type:      notifTypeEnum('type').notNull(),
  channel:   notifChannelEnum('channel').notNull(),
  status:    notifStatusEnum('status').default('pending'),
  title:     varchar('title', { length: 255 }).notNull(),
  body:      text('body').notNull(),
  data:      text('data'),         // JSON: deep link params e.g. { screen: 'narrative', id: '...' }
  readAt:    timestamp('read_at', { withTimezone: true }),
  sentAt:    timestamp('sent_at', { withTimezone: true }),
  errorMsg:  text('error_msg'),
  createdAt: timestamp('created_at', { withTimezone: true }).defaultNow(),
})

export const notificationPreferences = pgTable('notification_preferences', {
  // Same as userPreferences in user service — use user_preferences table there
  // notification-service reads from user_preferences via internal API call
})

// Card Shows
export const cardShows = pgTable('card_shows', {
```

```
  id:          uuid('id').primaryKey().defaultRandom(),
  name:        varchar('name', { length: 255 }).notNull(),
  venue:       varchar('venue', { length: 255 }),
  address:     text('address'),
  city:        varchar('city', { length: 100 }),
  state:       varchar('state', { length: 50 }),
  lat:         varchar('lat', { length: 20 }),
  lng:         varchar('lng', { length: 20 }),
  startDate:   timestamp('start_date', { withTimezone: true }).notNull(),
  endDate:     timestamp('end_date', { withTimezone: true }),
  website:     varchar('website', { length: 500 }),
  admission:   varchar('admission', { length: 50 }),
  description: text('description'),
  isActive:    boolean('is_active').default(true),
  createdAt:   timestamp('created_at', { withTimezone: true }).defaultNow(),
})

export const showAttendees = pgTable('show_attendees', {
  id:        uuid('id').primaryKey().defaultRandom(),
  showId:    uuid('show_id').references(() => cardShows.id, { onDelete: 'cascade' }).notNull(),
  userId:    uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).notNull(),
  role:      varchar('role', { length: 20 }).default('attendee'), // dealer | attendee
  createdAt: timestamp('created_at', { withTimezone: true }).defaultNow(),
})
```

9. Analytics Service — shared/db/schema/analytics.ts

```
import { pgTable, uuid, varchar, decimal, integer, timestamp, text } from 'drizzle-orm/pg-core'
import { users } from './auth'

// Pre-aggregated daily summaries — built by analytics-snapshot BullMQ job at 2am
export const dailySummaries = pgTable('daily_summaries', {
  id:               uuid('id').primaryKey().defaultRandom(),
  userId:           uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).notNull(),
  date:             varchar('date', { length: 10 }).notNull(),  // YYYY-MM-DD
  cardsBought:      integer('cards_bought').default(0),
  cardsSold:        integer('cards_sold').default(0),
  totalSpent:       decimal('total_spent', { precision: 10, scale: 2 }).default('0'),
  totalRevenue:     decimal('total_revenue', { precision: 10, scale: 2 }).default('0'),
  netProfit:        decimal('net_profit', { precision: 10, scale: 2 }).default('0'),
  bestDealMargin:   decimal('best_deal_margin', { precision: 8, scale: 2 }),
  revenueByChannel: text('revenue_by_channel'),  // JSON {ebay:200, card_show:150}
  revenueByPayment: text('revenue_by_payment'),  // JSON {cash:100, venmo:200}
  createdAt:        timestamp('created_at', { withTimezone: true }).defaultNow(),
}, (t) => ({
  uniqUserDate: uniqueIndex('uq_daily_summary_user_date').on(t.userId, t.date),
}))

export const taxRecords = pgTable('tax_records', {
  id:               uuid('id').primaryKey().defaultRandom(),
  userId:           uuid('user_id').references(() => users.id).notNull(),
  taxYear:          integer('tax_year').notNull(),
  totalRevenue:     decimal('total_revenue', { precision: 12, scale: 2 }),
  totalCostBasis:   decimal('total_cost_basis', { precision: 12, scale: 2 }),
  grossProfit:      decimal('gross_profit', { precision: 12, scale: 2 }),
```

```
  shortTermGains:   decimal('short_term_gains', { precision: 12, scale: 2 }),
  longTermGains:    decimal('long_term_gains', { precision: 12, scale: 2 }),
  platformFeesPaid: decimal('platform_fees_paid', { precision: 12, scale: 2 }),
  totalExpenses:    decimal('total_expenses', { precision: 12, scale: 2 }),
  reportS3Url:      varchar('report_s3_url', { length: 500 }),
  generatedAt:      timestamp('generated_at', { withTimezone: true }).defaultNow(),
})

export const expenses = pgTable('expenses', {
  id:          uuid('id').primaryKey().defaultRandom(),
  userId:      uuid('user_id').references(() => users.id).notNull(),
  category:    varchar('category', { length: 100 }),  // show_fee|travel|supplies|shipping
  description: varchar('description', { length: 255 }),
  amount:      decimal('amount', { precision: 10, scale: 2 }).notNull(),
  receiptUrl:  varchar('receipt_url', { length: 500 }),
  expenseDate: timestamp('expense_date', { withTimezone: true }).notNull(),
  createdAt:   timestamp('created_at', { withTimezone: true }).defaultNow(),
})
```

10. Admin Service — shared/db/schema/admin.ts

```
import { pgTable, uuid, varchar, text, boolean, timestamp, integer } from 'drizzle-orm/pg-core'
import { users } from './auth'

export const featureFlags = pgTable('feature_flags', {
  id:          uuid('id').primaryKey().defaultRandom(),
  key:         varchar('key', { length: 100 }).unique().notNull(), // aiNarratives, stripePayments
  value:       boolean('value').default(false),
  description: text('description'),
  updatedBy:   uuid('updated_by').references(() => users.id),
  updatedAt:   timestamp('updated_at', { withTimezone: true }).defaultNow(),
})

export const dealerReviews = pgTable('dealer_reviews', {
  id:         uuid('id').primaryKey().defaultRandom(),
  dealerId:   uuid('dealer_id').references(() => users.id).notNull(),
  reviewerId: uuid('reviewer_id').references(() => users.id).notNull(),
  rating:     integer('rating').notNull(),  // 1-5
  comment:    text('comment'),
  isApproved: boolean('is_approved').default(false),
  createdAt:  timestamp('created_at', { withTimezone: true }).defaultNow(),
})

export const auditLogs = pgTable('audit_logs', {
  id:         uuid('id').primaryKey().defaultRandom(),
  userId:     uuid('user_id').references(() => users.id),
  action:     varchar('action', { length: 255 }).notNull(),
  resource:   varchar('resource', { length: 100 }),
  resourceId: varchar('resource_id', { length: 255 }),
  ipAddress:  varchar('ip_address', { length: 50 }),
  userAgent:  text('user_agent'),
  metadata:   text('metadata'),  // JSON extra context
  createdAt:  timestamp('created_at', { withTimezone: true }).defaultNow(),
})
```

INDEXES

Complete DB Index Reference — All Tables, All Services

Run these after migrations. Critical for card show scale (5000+ concurrent users).

```
-- ================================================================
-- AUTH SERVICE INDEXES
-- ================================================================
CREATE UNIQUE INDEX idx_users_email        ON users(email);
CREATE        INDEX idx_users_role         ON users(role);
CREATE UNIQUE INDEX idx_refresh_token_hash ON refresh_tokens(token_hash);
CREATE        INDEX idx_refresh_user_id    ON refresh_tokens(user_id);
CREATE        INDEX idx_device_tokens_user ON device_tokens(user_id);

-- ================================================================
-- USER SERVICE INDEXES
-- ================================================================
CREATE UNIQUE INDEX idx_dealer_profile_user   ON dealer_profiles(user_id);
CREATE UNIQUE INDEX idx_consumer_profile_user ON consumer_profiles(user_id);
CREATE UNIQUE INDEX idx_dealer_custom_url      ON dealer_profiles(custom_url);
CREATE        INDEX idx_payment_methods_user   ON payment_methods(user_id);
CREATE        INDEX idx_platform_conn_user     ON platform_connections(user_id, platform);
CREATE UNIQUE INDEX idx_user_preferences_user  ON user_preferences(user_id);
CREATE        INDEX idx_dealer_followers_dealer ON dealer_followers(dealer_id);
CREATE        INDEX idx_dealer_followers_follower ON dealer_followers(follower_id);
CREATE        INDEX idx_customers_user         ON customers(user_id);

-- ================================================================
-- INVENTORY SERVICE INDEXES  (HIT ON EVERY DEALER SCREEN)
-- ================================================================
CREATE INDEX idx_inventory_user_id        ON inventory(user_id);
CREATE INDEX idx_inventory_listing_status ON inventory(listing_status);
CREATE INDEX idx_inventory_card_id        ON inventory(card_id);
CREATE INDEX idx_inventory_added_at       ON inventory(added_at);        -- for aging alerts
CREATE INDEX idx_inventory_user_status    ON inventory(user_id, listing_status);  -- composite
CREATE INDEX idx_inventory_player         ON inventory(player_name);     -- for want list match

-- ================================================================
-- TRANSACTION SERVICE INDEXES  (HIT ON EVERY REPORT QUERY)
-- ================================================================
CREATE INDEX idx_transactions_user_created ON transactions(user_id, created_at DESC);  -- MOST IMPORTANT
CREATE INDEX idx_transactions_type         ON transactions(type);
CREATE INDEX idx_transactions_channel      ON transactions(channel);
CREATE INDEX idx_transactions_customer     ON transactions(customer_id);
CREATE UNIQUE INDEX idx_transactions_local_id ON transactions(local_id); -- dedup offline sync
CREATE INDEX idx_transactions_user_type_date ON transactions(user_id, type, created_at DESC);
CREATE INDEX idx_trade_items_transaction   ON trade_items(transaction_id);

-- ================================================================
-- LISTING SERVICE INDEXES
-- ================================================================
CREATE INDEX idx_listings_inventory_id    ON listings(inventory_id, status);
CREATE INDEX idx_listings_user_id         ON listings(user_id, status);
CREATE INDEX idx_listings_platform_id     ON listings(platform_listing_id, platform);  -- webhook lookup
```

```
CREATE INDEX idx_listings_status          ON listings(status);

-- ================================================================
-- CARD DB SERVICE INDEXES  (HIT ON EVERY BUY FLOW SCAN)
-- ================================================================
CREATE        INDEX idx_cards_player_name    ON cards(player_name);
CREATE        INDEX idx_cards_sport          ON cards(sport);
CREATE        INDEX idx_cards_year_set       ON cards(year, set_name);

-- platformSoldListings: hit every time comps are fetched
CREATE        INDEX idx_sold_card_grade      ON platform_sold_listings(card_id, grade_key);
CREATE        INDEX idx_sold_platform_date   ON platform_sold_listings(platform, sold_at DESC);
CREATE        INDEX idx_sold_card_grade_plat ON platform_sold_listings(card_id, grade_key, platform);
CREATE UNIQUE INDEX idx_sold_content_hash    ON platform_sold_listings(content_hash);   -- dedup

-- cardCompSnapshots: hit on every BUY screen load
CREATE UNIQUE INDEX idx_comp_card_grade_plat ON card_comp_snapshots(card_id, grade_key, platform);
CREATE        INDEX idx_comp_fetched_at      ON card_comp_snapshots(fetched_at);

-- cardPriceHistory: hit on chart render
CREATE INDEX idx_price_hist_card_grade ON card_price_history(card_id, grade_key, recorded_date DESC);

-- priceAlerts: hit every 15-minute evaluation cron
CREATE INDEX idx_price_alerts_user    ON price_alerts(user_id);
CREATE INDEX idx_price_alerts_active  ON price_alerts(is_active, is_triggered);
CREATE INDEX idx_price_alerts_card    ON price_alerts(card_id, grade_key);

-- wantList: hit when dealer adds card (match check)
CREATE INDEX idx_want_list_user       ON want_list(user_id);
CREATE INDEX idx_want_list_card_grade ON want_list(card_id, grade_key);

-- consumerCollection: hit on portfolio load
CREATE INDEX idx_collection_user      ON consumer_collection(user_id);
CREATE INDEX idx_collection_card      ON consumer_collection(card_id);

-- ================================================================
-- AI NARRATIVE SERVICE INDEXES
-- ================================================================
CREATE UNIQUE INDEX idx_watchlist_player    ON player_watchlist(player_name);
CREATE        INDEX idx_watchlist_tier      ON player_watchlist(tier, active);
CREATE        INDEX idx_watchlist_active    ON player_watchlist(active);

CREATE UNIQUE INDEX idx_snapshot_player     ON player_snapshots(player_name);
CREATE        INDEX idx_snapshot_fetched    ON player_snapshots(last_fetched_at);

CREATE INDEX idx_snapshot_hist_player ON player_snapshot_history(player_name, created_at DESC);

CREATE INDEX idx_narratives_player  ON narratives(player_name, published_at DESC);
CREATE INDEX idx_narratives_status  ON narratives(status);
CREATE INDEX idx_narratives_type    ON narratives(narrative_type);

-- ================================================================
-- NOTIFICATION SERVICE INDEXES
-- ================================================================
```

```
CREATE INDEX idx_notifications_user   ON notifications(user_id, created_at DESC);
CREATE INDEX idx_notifications_status ON notifications(status);
CREATE INDEX idx_notifications_unread ON notifications(user_id, status) WHERE status = 'pending';
CREATE INDEX idx_shows_start_date     ON card_shows(start_date);
CREATE INDEX idx_shows_city           ON card_shows(city, state);
CREATE INDEX idx_attendees_show       ON show_attendees(show_id);
CREATE INDEX idx_attendees_user       ON show_attendees(user_id);

-- ================================================================
-- ANALYTICS SERVICE INDEXES
-- ================================================================
CREATE UNIQUE INDEX idx_daily_sum_user_date ON daily_summaries(user_id, date);
CREATE        INDEX idx_tax_records_user    ON tax_records(user_id, tax_year);
CREATE        INDEX idx_expenses_user       ON expenses(user_id, expense_date);

-- ================================================================
-- ADMIN SERVICE INDEXES
-- ================================================================
CREATE UNIQUE INDEX idx_feature_flags_key   ON feature_flags(key);
CREATE        INDEX idx_dealer_reviews_dealer ON dealer_reviews(dealer_id, is_approved);
CREATE        INDEX idx_audit_logs_user     ON audit_logs(user_id, created_at DESC);
CREATE        INDEX idx_audit_logs_action   ON audit_logs(action, created_at DESC);
```

APIS

Complete API Endpoint Reference — All 10 Services

Every endpoint with method, route, which service handles it, when BullMQ fires

Auth Service APIs (handled locally by auth-service, NOT proxied)

Method	Route	Description	BullMQ?
POST	/v1/auth/register	Register with email+password. Calls user-service /internal/users/create after success.	No
POST	/v1/auth/login	Login. Returns tokens or requires_2fa + temp_token if 2FA enabled.	No
POST	/v1/auth/refresh	Token rotation. Old refresh token invalidated. New pair returned.	No
POST	/v1/auth/logout	Add JTI to Redis blocklist. Delete refresh token from DB.	No
POST	/v1/auth/oauth/google	Verify Google ID token. Find or create user. Return tokens.	No
POST	/v1/auth/oauth/apple	Verify Apple identity token via JWKS. Find or create user.	No
POST	/v1/auth/2fa/setup	Generate TOTP secret + QR code. Store encrypted secret.	No
POST	/v1/auth/2fa/verify	Verify 6-digit TOTP code. Activate 2FA or complete login.	No
POST	/v1/auth/2fa/disable	Verify password + TOTP, then disable 2FA.	No
POST	/v1/auth/device-token	Store FCM device token for push notifications.	No
DELETE	/v1/auth/device-token	Remove FCM token on logout.	No
GET	/v1/health	Real DB + Redis ping. Used by Docker, admin-service.	No
GET	/v1/internal/probe	Service mesh health check (X-Service-Key required).	No

User Service APIs (/v1/users/* — proxied through auth-gateway)

Method	Route	Description	BullMQ?
GET	/v1/users/me	Get own full profile + plan + stats.	No
PATCH	/v1/users/me	Update profile fields.	No
GET	/v1/users/:id/profile	Get dealer public profile (for consumer app dealer discovery).	No
GET	/v1/users/dealers	List dealers. Query: sport, lat, lng, radius, sort.	No
POST	/v1/users/platforms/:name/connect	Connect eBay/Whatnot OAuth. Stores encrypted tokens.	No
DELETE	/v1/users/platforms/:name	Disconnect platform. Revoke tokens.	No
GET	/v1/users/me/preferences	Get notification + display preferences.	No
PATCH	/v1/users/me/preferences	Update preferences.	No
GET	/v1/users/customers	List dealer's contacts.	No
POST	/v1/users/customers	Add customer to dealer contact list.	No
PATCH	/v1/users/customers/:id	Update customer notes, favorite.	No
DELETE	/v1/users/customers/:id	Remove customer.	No
POST	/v1/users/me/follow/:dealerId	Follow a dealer (consumer).	No
DELETE	/v1/users/me/follow/:dealerId	Unfollow dealer.	No

Inventory Service APIs (/v1/inventory/*)

Method	Route	Description	BullMQ?
GET	/v1/inventory	Paginated list. Query: sport, status, gradeCompany, sort, page.	No
POST	/v1/inventory	Add card manually. Fires want-list-match queue.	PRODUCER: want-list-match
GET	/v1/inventory/:id	Single card detail with comps + chart + narratives.	No
PATCH	/v1/inventory/:id	Edit card (price, notes, photos, grade).	No
DELETE	/v1/inventory/:id	Archive/remove card from inventory.	No
POST	/v1/inventory/bulk-import	CSV upload with AI column mapping.	No
GET	/v1/inventory/summary	Total cards, cost basis, market value, unrealized gain.	No
GET	/v1/inventory/aging-alerts	Cards held 60+ days.	No
POST	/v1/inventory/bulk-purchase	Record bulk lot purchase with proportional cost allocation.	No
POST	/v1/inventory/consignment	Add consignment card with owner + commission.	No

Transaction Service APIs (/v1/transactions/*)

Method	Route	Description	BullMQ?
POST	/v1/transactions/buy	Record BUY. Creates inventory record. Returns deal_rating.	No
POST	/v1/transactions/sell	Record SELL. Fires deactivate-listings + sale-notification.	PRODUCER: deactivate-listings, sale-notification
POST	/v1/transactions/trade	Record TRADE. Updates inventory both ways.	No
POST	/v1/transactions/sync	Drain offline queue. Batch process. dedup by localId.	No
GET	/v1/transactions	History. Query: type, channel, dateFrom, dateTo, page.	No
GET	/v1/transactions/:id	Single transaction detail.	No
GET	/v1/transactions/today	Today stats: bought, sold, spent, revenue, profit.	No
GET	/v1/transactions/export	Export as CSV for date range.	No
DELETE	/v1/transactions/:id	Void transaction (with reason log).	No

Listing Service APIs (/v1/listings/*)

Method	Route	Description	BullMQ?
GET	/v1/listings	All active listings across all platforms.	No
POST	/v1/listings	Create listing on one or more platforms simultaneously.	No
GET	/v1/listings/:id	Single listing with platform status.	No
PATCH	/v1/listings/:id/price	Update price on active listing (calls platform API).	No
DELETE	/v1/listings/:id	End listing on platform.	No

Method	Route	Description	BullMQ?
POST	/v1/listings/:id/relist	Relist ended listing.	No
GET	/v1/listings/price-comparison/:invId	Cross-platform comp prices for a card (reads cardCompSnapshots).	No
GET	/v1/listings/fee-calculator	Net profit per platform for given price.	No
POST	/v1/listings/generate-content	AI-generate title + description for listing.	No
GET	/v1/listings/analytics	Platform performance: views, watchers, sell-through.	No
POST	/v1/listings/webhooks/ebay	eBay sold webhook. Fires deactivate-listings queue.	PRODUCER: deactivate-listings, sale-notification
POST	/v1/listings/webhooks/whatnot	Whatnot sold webhook.	PRODUCER: deactivate-listings, sale-notification
POST	/v1/listings/webhooks/mercari	Mercari sold webhook.	PRODUCER: deactivate-listings, sale-notification
POST	/v1/listings/webhooks/tcgplayer	TCGPlayer sold webhook.	PRODUCER: deactivate-listings, sale-notification
POST	/v1/listings/webhooks/shopify	Shopify order webhook.	PRODUCER: deactivate-listings, sale-notification

Card DB Service APIs (/v1/cards/*)

Method	Route	Description	BullMQ?
POST	/v1/cards/scan	Image scan via Ximilar. Returns card + graded comps from cardCompSnapshots.	No
POST	/v1/cards/scan/barcode	Graded card cert barcode lookup.	No
GET	/v1/cards/search	Text search: player, year, set. Returns matches.	No
GET	/v1/cards/:id	Card detail + latest comps.	No
GET	/v1/cards/:id/comps	Last 5 sold from platformSoldListings + avg + trend. gradeKey param required.	No
GET	/v1/cards/:id/comps-by-platform	Per-platform snapshot from cardCompSnapshots. Powers BUY screen comparison.	No
GET	/v1/cards/:id/price-history	30/90/365 day chart data from cardPriceHistory. gradeKey param required.	No
GET	/v1/cards/offline-db	Download compressed 50K card offline cache (S3 signed URL).	No
GET	/v1/cards/price-alerts	Get user's active price alerts.	No
POST	/v1/cards/price-alerts	Create price alert. gradeKey required.	No
PATCH	/v1/cards/price-alerts/:id	Update or pause/unpause alert.	No
DELETE	/v1/cards/price-alerts/:id	Delete price alert.	No
GET	/v1/cards/want-list	Get user's want list.	No
POST	/v1/cards/want-list	Add to want list with gradeKey + maxPrice.	No

Method	Route	Description	BullMQ?
DELETE	/v1/cards/want-list/:id	Remove from want list.	No
GET	/v1/cards/deal-rating	Get deal rating for price vs comp. Query: cardId, gradeKey, price.	No

AI Narrative Service APIs (/v1/narratives/*)

Method	Route	Description	BullMQ?
GET	/v1/narratives/feed	Market movers feed — latest published narratives (consumer home).	No
GET	/v1/narratives/inventory	Narratives relevant to dealer's current inventory.	No
GET	/v1/narratives/:id	Full narrative detail.	No
GET	/v1/narratives/player/:playerName	All narratives for a player.	No
GET	/v1/narratives/card/:cardId	Why is this card moving? Narratives for specific card.	No
GET	/v1/narratives/daily-insight	Single top daily insight for dealer home screen.	No
GET	/v1/narratives/weekly-recap	AI weekly recap of collection performance.	No
POST	/v1/narratives/trigger-ingestion	DEV ONLY: manually fire ingestion cycle.	PRODUCER: data-ingestion
PATCH	/v1/narratives/admin/:id/approve	Approve narrative. Fires ai-narrative-publish queue.	PRODUCER: ai-narrative-publish
PATCH	/v1/narratives/admin/:id/reject	Reject narrative with reason.	No
PATCH	/v1/narratives/admin/:id	Edit narrative before approving.	No
POST	/v1/narratives/admin/generate	Manually trigger narrative for a player.	PRODUCER: narrative-generate

Notification Service APIs (/v1/notifications/* and /v1/shows/*)

Method	Route	Description	BullMQ?
GET	/v1/notifications	In-app notifications (unread first, paginated).	No
PATCH	/v1/notifications/:id/read	Mark single notification read.	No
PATCH	/v1/notifications/read-all	Mark all notifications read.	No
GET	/v1/notifications/unread-count	Count of unread (for bell badge).	No
GET	/v1/shows	Upcoming shows. Query: lat, lng, radius, dateFrom.	No
GET	/v1/shows/:id	Show detail + dealers + want list matches.	No
POST	/v1/shows/:id/attend	Mark attending (dealer or consumer).	No
DELETE	/v1/shows/:id/attend	Remove attendance.	No
GET	/v1/shows/:id/dealers	Dealers attending with public inventory preview.	No
POST	/v1/shows/admin	Admin: create card show. (Admin role required)	No
PATCH	/v1/shows/admin/:id	Admin: update show details.	No

Analytics Service APIs (/v1/analytics/*)

Method	Route	Description	BullMQ?
GET	/v1/analytics/daily	Today's full stats. Redis cached 5min.	No
GET	/v1/analytics/report	Period report. Query: period=week|month|custom, dateFrom, dateTo. Read replica.	No
GET	/v1/analytics/profit-by-sport	Revenue + profit by sport. Read replica.	No
GET	/v1/analytics/profit-by-channel	Revenue by channel (eBay, card_show etc). Read replica.	No
GET	/v1/analytics/top-cards	Top 5 most profitable + top 5 worst performers.	No
GET	/v1/analytics/inventory-value-trend	Portfolio value trend over time (chart data). Read replica.	No
GET	/v1/analytics/platform-performance	Sales, revenue, avg margin per listing platform.	No
GET	/v1/analytics/tax/:year	Full tax report for year: P&L;, gains, Schedule C.	No
POST	/v1/analytics/tax-export	Queue async tax export job. Returns jobId immediately.	PRODUCER: tax-report-generate
GET	/v1/analytics/tax-export/:jobId	Check export status. Returns S3 download URL when done.	No
GET	/v1/analytics/export	Export transactions as CSV for date range.	No
GET	/v1/analytics/expenses	List tracked expenses.	No
POST	/v1/analytics/expenses	Add expense record.	No
PATCH	/v1/analytics/expenses/:id	Update expense.	No
DELETE	/v1/analytics/expenses/:id	Delete expense.	No
GET	/v1/analytics/collection	Consumer portfolio: total value, gain/loss, best/worst.	No
GET	/v1/analytics/trigger-snapshot	DEV ONLY: manually fire daily snapshot job.	PRODUCER: analytics-snapshot

Admin Service APIs (/v1/admin/*)

Method	Route	Description	BullMQ?
GET	/v1/admin/users	List all users with role, status, join date, stats.	No
GET	/v1/admin/users/:id	Full user detail.	No
PATCH	/v1/admin/users/:id/role	Change user role.	No
PATCH	/v1/admin/users/:id/suspend	Suspend account.	No
PATCH	/v1/admin/users/:id/unsuspend	Restore account.	No
DELETE	/v1/admin/users/:id	Permanently delete user + all data.	No
GET	/v1/admin/narratives/pending	Narratives pending review.	No
PATCH	/v1/admin/narratives/:id/approve	Approve. Fires ai-narrative-publish queue.	PRODUCER: ai-narrative-publish
PATCH	/v1/admin/narratives/:id/reject	Reject with reason.	No
PATCH	/v1/admin/narratives/:id	Edit before approving.	No
GET	/v1/admin/feature-flags	All feature flags.	No

Method	Route	Description	BullMQ?
PATCH	/v1/admin/feature-flags/:key	Toggle feature on/off.	No
GET	/v1/admin/reviews/pending	Reviews pending approval.	No
PATCH	/v1/admin/reviews/:id/approve	Approve dealer review.	No
DELETE	/v1/admin/reviews/:id	Remove inappropriate review.	No
GET	/v1/admin/health-all	Pings /health on all 9 services. Returns aggregate.	No
GET	/v1/admin/audit-logs	System audit log.	No
GET	/v1/admin/stats	Platform stats: total users, DAU, transactions.	No
GET	/v1/config/feature-flags	Public feature flags for mobile app config. (JWT)	No